

NiceGUI ui.keyboard 全面详解

ui.keyboard 是 NiceGUI 提供的全局键盘事件跟踪组件，支持监听键盘按键的按下、释放、重复触发等事件，可通过灵活配置实现键盘交互逻辑，同时提供事件拦截、属性绑定等高级功能，适用于全局快捷键、键盘导航等场景。

一、核心概念与事件模型

1. 核心功能定位

- 全局监听：无需绑定特定 DOM 元素，可捕获整个应用的键盘事件
- 事件粒度：支持按键按下 (keydown)、释放 (keyup)、重复触发 (repeat) 三种行为监听
- 灵活控制：可启用 / 禁用事件监听、忽略特定元素聚焦时的键盘事件、控制长按重复触发等

2. 事件参数结构 (KeyEventArguments)

键盘事件触发时，回调函数会接收 `KeyEventArguments` 对象，包含以下核心属性：

属性	类型	说明
sender	Keyboard 实例	触发事件的键盘组件本身
client	Client 对象	关联的客户端实例
action	KeyboardAction 对象	按键行为信息 (按下 / 释放 / 重复)
key	KeyboardKey 对象	按键本身信息 (名称、编码、位置等)
modifiers	KeyboardModifiers 对象	修饰键状态 (Ctrl/Shift/Alt/Meta 等)

(1) KeyboardAction 详解

描述按键的行为状态，属性均为布尔值：

- `keydown`：是否为按下事件
- `keyup`：是否为释放事件
- `repeat`：是否为长按重复触发的事件

(2) KeyboardKey 详解

描述按键的基础信息，包含核心属性和便捷属性：

- 核心属性：
 - `name`：按键名称 (如 "a"、"Enter"、"ArrowLeft"，完整列表参考[MDN 按键名称](#))
 - `code`：按键编码 (如 "KeyA"、"Enter"、"ArrowLeft"，与物理按键位置绑定)
 - `location`：按键位置 (0 = 标准键、1 = 左键、2 = 右键、3 = 小键盘键)
- 便捷属性 (布尔值，快速判断按键类型)：
 - 方向键：`arrow_left` / `arrow_up` / `arrow_right` / `arrow_down`
 - 功能键：`f1` - `f12`、`escape`、`tab`、`enter`、`space` 等

- 编辑键: `backspace`、`insert`、`delete`、`page_up`、`page_down` 等
- 状态键: `caps_lock`、`pause`、`print_screen` 等
- 辅助属性: `is_cursorkey` (是否为方向键)、`number` (数字键的整数值, 非数字键返回 `None`)

(3) KeyboardModifiers 详解

描述修饰键的按下状态, 属性均为布尔值:

- `alt`: Alt 键 (Windows) /Option 键 (Mac)
- `ctrl`: Ctrl 键
- `meta`: Meta 键 (Windows 键 / Command 键)
- `shift`: Shift 键

二、初始化参数配置

创建 `ui.keyboard` 时可通过参数控制核心行为, 默认值已适配常见场景:

参数名	类型	默认值	说明
<code>on_key</code>	<code>Callable</code>	-	键盘事件触发时执行的回调函数, 接收 <code>KeyEventArguments</code> 参数
<code>active</code>	<code>bool</code>	<code>True</code>	是否启用事件回调 (可动态绑定修改)
<code>repeating</code>	<code>bool</code>	<code>True</code>	长按按键时是否重复触发事件
<code>ignore</code>	<code>List[str]</code>	<code>['input', 'select', 'button', 'textarea']</code>	当这些类型的元素获得焦点时, 忽略键盘事件 (避免干扰输入框等原生交互)

示例: 基础初始化

```
from nicegui import ui
from nicegui.events import KeyEventArguments

def handle_key(e: KeyEventArguments):
    print(f"按键: {e.key.name}, 行为: {'按下' if e.action.keydown else '释放'}, 修饰键: {e.modifiers.shift}")

# 创建键盘组件并绑定回调
ui.keyboard(on_key=handle_key)
ui.run()
```

三、核心功能与使用示例

1. 基础按键监听

支持精准监听特定按键的按下 / 释放事件，结合修饰键判断：

```
from nicegui import ui
from nicegui.events import KeyEventArguments

def handle_key(e: KeyEventArguments):
    # 监听 F 键的按下和释放（忽略重复触发）
    if e.key == 'f' and not e.action.repeat:
        if e.action.keydown:
            ui.notify('F键被按下')
        elif e.action.keyup:
            ui.notify('F键被释放')

    # 监听 Shift+方向键组合
    if e.modifiers.shift and e.action.keydown:
        if e.key.arrow_left:
            ui.notify('Shift+左箭头')
        elif e.key.arrow_right:
            ui.notify('Shift+右箭头')

ui.keyboard(on_key=handle_key)
ui.label('按下 F 键或 Shift+方向键测试')
ui.run()
```

2. 动态控制事件监听（active 绑定）

通过 `bind_value_to` 可将 `active` 属性与其他组件绑定，实现动态启用 / 禁用监听：

```
from nicegui import ui
from nicegui.events import KeyEventArguments

def handle_key(e: KeyEventArguments):
    if e.key.enter and e.action.keydown:
        ui.notify('Enter键被按下')

keyboard = ui.keyboard(on_key=handle_key)
# 复选框绑定 active 属性，控制是否监听键盘事件
ui.checkbox('启用键盘监听', value=True).bind_value_to(keyboard, 'active')
ui.run()
```

3. 阻止默认行为与事件传播

通过 `on` 方法的 `js_handler` 参数，可在客户端拦截事件，阻止浏览器默认行为（如禁用 Ctrl+A 选择）：

```
from nicegui import ui

ui.label('Ctrl+A / Cmd+A 选择功能已禁用')
```

```
# 拦截 Ctrl+A/Cmd+A 事件
ui.keyboard().on(
    'key',
    lambda: ui.notify('选择功能已被禁用'),
    js_handler='''(e) => {
        // 判断按键为 A 且按下 Ctrl 或 Meta 键
        if (e.key === 'a' && (e.ctrlKey || e.metaKey) && e.action === 'keydown') {
            emit(e); // 触发Python回调（可选）
            e.event.preventDefault(); // 阻止默认选择行为
            e.event.stopPropagation(); // 阻止事件传播
        }
    }'''
)

ui.run()
```

注：该功能需 NiceGUI 3.1.0+ 版本支持

4. 忽略特定元素的键盘事件

默认情况下，输入框、按钮等元素聚焦时会忽略全局键盘事件，可通过 `ignore` 参数自定义忽略列表：

```
# 仅在输入框聚焦时忽略全局键盘事件
ui.keyboard(ignore=['input'], on_key=lambda e: ui.notify(f'按下: {e.key.name}'))
ui.input('输入框聚焦时，全局键盘事件会被忽略')
ui.button('按钮聚焦时，全局键盘事件仍生效')
ui.run()
```

5. 禁用长按重复触发

通过 `repeating=False` 禁用长按按键时的重复事件触发：

```
# 长按按键不会重复触发回调
ui.keyboard(repeating=False, on_key=lambda e: ui.notify(f'按键: {e.key.name}'))
ui.label('长按按键不会重复触发事件')
ui.run()
```

四、组件属性与方法

1. 核心属性（可绑定 / 访问）

属性名	类型	说明
active	BindableProperty	控制事件回调是否启用（支持双向绑定）
classes	str	组件的 HTML 类（用于样式定制）
html_id	str	组件的 HTML DOM ID（3.16.0 + 版本支持）
is_deleted	bool	组件是否已被删除

属性名	类型	说明
is_ignoring_events	bool	当前是否忽略事件（根据 ignore 参数和聚焦状态动态判断）
visible	BindableProperty	控制组件是否可见（仅影响 DOM 显示，不影响事件监听）

2. 常用方法

方法名	参数说明	功能描述
on_key(handler)	handler: 回调函数（接收 KeyEventArguments 或无参数）	追加键盘事件回调
bind_visibility(target_object, ...)	目标对象、属性名等（见参数表）	双向绑定组件可见性到目标对象的属性
bind_visibility_from(...)	同 bind_visibility，但仅单向从目标对象同步	单向绑定可见性（目标→组件）
bind_visibility_to(...)	同 bind_visibility，但仅单向从组件同步	单向绑定可见性（组件→目标）
delete()	无	删除组件及所有子元素
set_visibility(visible)	visible: bool	直接设置组件可见性
tooltip(text)	text: 提示文本	为组件添加悬浮提示
update()	无	强制更新组件在客户端的状态

示例：属性绑定与方法使用

```
from nicegui import ui

keyboard = ui.keyboard(on_key=lambda e: ui.notify(f'按下: {e.key.name}'))

# 绑定可见性到复选框
visible_checkbox = ui.checkbox('显示组件', value=True)
keyboard.bind_visibility(visible_checkbox, 'value')

# 按钮控制active状态
ui.button('禁用监听', on_click=lambda: setattr(keyboard, 'active', False))
ui.button('启用监听', on_click=lambda: setattr(keyboard, 'active', True))

ui.run()
```

五、高级用法：事件节流与自定义 JS 处理

1. 事件节流（避免高频触发）

通过 `on` 方法的 `throttle` 参数限制事件触发频率（适用于高频事件如方向键连续按下）：

```
from nicegui import ui

# 限制1秒内最多触发1次事件
ui.keyboard().on(
    'key',
    lambda e: ui.notify(f'方向键: {e.key.name}'),
    throttle=1.0, # 节流时间（秒）
    js_handler='''(e) => {
        if (e.key.is_cursorkey) emit(e); // 仅方向键触发
    }'''
)
ui.label('方向键事件1秒内最多触发1次')
ui.run()
```

2. 自定义 JS 处理逻辑

通过 `js_handler` 实现客户端预处理，仅在满足条件时向服务器发送事件（减少网络通信）：

```
from nicegui import ui

ui.keyboard().on(
    'key',
    lambda: ui.notify('Ctrl+S 保存成功'), # Python回调（服务器端）
    js_handler='''(e) => {
        // 客户端判断：仅 Ctrl+S 按下时触发
        if (e.key === 's' && e.ctrlKey && e.action === 'keydown') {
            e.event.preventDefault(); // 阻止浏览器默认保存行为
            emit(e); // 触发Python回调
        }
    }'''
)
ui.label('按下 Ctrl+S 测试')
ui.run()
```

六、版本兼容性与注意事项

1. 版本要求

- 基础功能：支持 NiceGUI 2.0+
- 阻止默认行为 (`js_handler`)：需 3.1.0+ 版本
- `html_id` 属性：需 3.16.0+ 版本

2. 注意事项

- 事件优先级：原生元素（如输入框）的键盘事件优先级高于全局监听，符合用户直觉
- 跨平台兼容性：`meta` 键自动适配 Windows（Windows 键）和 Mac（Command 键）
- 按键名称大小写：`e.key.name` 区分大小写（如 "a" 是小写，"A" 需结合 shift 修饰键）
- 性能优化：高频事件（如方向键导航）建议使用 `throttle` 参数或客户端 JS 处理，减少服务器压力

总结

`ui.keyboard` 是 NiceGUI 中功能强大的全局键盘交互组件，通过清晰的事件模型、灵活的配置参数和丰富的高级功能，可满足从简单快捷键到复杂键盘导航的各类需求。其核心优势在于全局监听无死角、事件控制精细化、与其他组件的绑定能力强，同时兼顾了原生交互兼容性和性能优化，是构建键盘优先的 Web 应用的理想选择。